Spintra — Production Readiness Audit

Scope: full repository (`Spintra-1`). Read-only audit — no code was changed as part of this report.

1. Executive Summary

Spintra is a party/game-room web app (11 standalone single-player tools + a Supabase-backed multiplayer room feature) built on Next.js 16 / React 19 with a clean, consistent shadcn-style design system. The UI layer, single-player tool logic, and component architecture are largely well-built. However, the product has **no real backend**: there is no API layer, no committed database schema/RLS, and the entire multiplayer trust model (identity, host status, room locking) is enforced client-side via `localStorage` and is trivially bypassable from devtools. Combine that with zero CI/CD, ~1 automated test covering a fraction of the app, a boilerplate README with no environment/setup docs, a broken double-elimination bracket, a visible FOUC theming bug, and use of a deprecated Next.js 16 file convention (`middleware.ts` vs `proxy.ts`) that risks silently breaking the room-join flow — this is not ready for a public, multi-user launch.

2. Overall Project Health Score: **48/100**

3. Production Readiness Score: **24/100**

Dimension	Score	Rationale
Code Quality	60/100	Clean patterns, but real logic bugs (broken bracket, weak PRNG, dead toggle)
Architecture	38/100	No API/server layer; auth/authz has no verifiable server enforcement; fake room persistence
Security	28/100	Client-side-only authorization, no headers, no rate limiting, unverifiable RLS
Performance	65/100	Fine at party-room scale; unbounded state and un-split 3D bundle are the main risks
Accessibility	55/100	Strong primitive layer (Base UI) undermined by missing `aria-pressed`/focus-visibility gaps
UX	55/100	Silent failures, misleading "Live" indicator, FOUC on every load
UI	72/100	Consistent design system, good theming foundation, some leaked one-off CSS
Testing	8/100	One Playwright smoke test; zero coverage of 10 of 11 tools and all realtime logic
Maintainability	50/100	Duplicated logic across 11 tool pages, inconsistent/stale lint artifacts committed to git
Documentation	8/100	README is unmodified `create-next-app` boilerplate; no env docs at all

4. Strengths

- **Design system**: consistent `cva`-based variants (`button.tsx`, `badge.tsx`, `tabs.tsx`), shared `cn()` utility, `data-slot` convention across all UI primitives.

- **Accessible primitives**: dialog/sheet/popover/dropdown/select/tooltip/accordion/tabs/switch/slider all wrap `@base-ui/react` (and `vaul` for drawer), inheriting real focus-trapping and keyboard handling rather than reinventing it.
- **Client resilience**: `getSupabaseBrowserClient()` returning `null` when unconfigured is checked consistently at every call site in `room-client.tsx` — no crash on missing env vars.
- **Channel cleanup**: Supabase channel and `BroadcastChannel` subscriptions are properly torn down on unmount — no realtime memory leaks found.
- **Single-player game logic**: Coin flip, dice, RPS, Name Draw, Guess-the-Number, and Tournament (single-elim/round-robin) all use correct, unbiased RNG and sound Fisher-Yates shuffling; Name Draw's opt-in (not automatic) localStorage save is a thoughtful UX choice.
- **No SQL-injection surface**: all Supabase calls use the parameterized query builder; zero string-concatenated queries found.
- **No secrets in repo**: no hardcoded keys/tokens, no committed `.env` files, `.gitignore` correctly excludes them.
- **Version integrity**: declared and installed `next` / `react` versions match exactly — no dependency drift.

---

5. Broken Features

#	Severity	Feature	File(s)	Evidence
1	Critical	Double-elimination tournament	`src/app/tools/tournament/page.tsx` (~556-574)	Losers-bracket shape is generated but `handleScoreSave` only advances winners in the winners bracket; losers are never fed into the losers bracket. Double-elimination mode is non-functional as designed.
2	Critical	Room creation never persists to Supabase	`src/app/create/create-client.tsx:36-53`	`handleCreate` only writes to `localStorage` (room type/name); no `insert` into a `rooms` table, no collision check against existing codes.
3	High	Participant "kick/manage" control	`src/app/room/[code]/room-client.tsx:1027-1031`	`MoreHorizontal` button renders with no `onClick` handler — dead UI.
4	Medium	Room capacity (`max_participants`)	`src/lib/types.ts` , `room-client.tsx`	Field exists in the `Room` type but is never read or enforced anywhere — rooms cannot actually be capped.
5	Medium	Room lock	`room-client.tsx` (`isLocked`)	Only gates `sendMessage` ; does not prevent new participants from joining a "locked" room.

6. Partially Implemented Features

- **Theming**: `next-themes` is a declared dependency but never used — a hand-rolled `ThemeProvider` reimplements it without the no-flash script, causing a real FOUC bug (`src/app/layout.tsx:54` hardcodes `className="dark"` ; actual theme is applied client-side post-mount).

- **Realtime "Live" status**: When Supabase isn't configured, the app silently falls back to a same-browser-only `BroadcastChannel` and still displays a "Live"/connected badge — misleading, since it will not sync across devices/users at all in that mode.
- **Host election**: `electHostIfNeeded` can be triggered from multiple concurrent event handlers with no compare-and-swap — simultaneous host claims are possible under race conditions.
- **Team Maker "Auto-balance"**: toggle exists in the UI and claims to control even distribution, but `generateTeams` applies the same distribution logic regardless of the toggle's state — the switch is a no-op.
- **Would You Rather / Never Have I Ever**: content cycles via simple modulo increment with no shuffling — every session repeats the same fixed order.

7. Missing Features

- No API/server layer at all (no `src/app/api`, no server actions) — everything is client-to-Supabase.
- No SQL migrations / RLS policy files in-repo — database security posture is entirely unverifiable from source.
- No CI/CD pipeline (no `.github/workflows`, no `vercel.json`).
- No rate limiting anywhere (room creation, chat, join-by-code brute force).
- No security headers (CSP, X-Frame-Options, HSTS, X-Content-Type-Options, Referrer-Policy) in `middleware.ts` or `next.config.ts`.
- No route-level error handling: zero `loading.tsx`, `error.tsx`, or `not-found.tsx` files anywhere in `src/app`.
- No `.env.example` or documented environment variables (Supabase URL/anon key needed to run the app at all).
- No unit/integration test framework — Playwright only, and only one smoke spec.
- No `typecheck` script — type errors are only caught implicitly during `next build`.
- No shared hooks/utilities for logic duplicated 11× (sound toggle, shuffle).

8. Bugs

Severity	Bug	File:Line
Critical	`middleware.ts` uses the file convention Next.js 16 deprecated in favor of `proxy.ts` (per `node_modules/next/dist/docs`); if the legacy convention silently stops executing, the entire `/room?code=X` shared-link join flow breaks with no fallback other than a duplicated client-side redirect in `room/page.tsx`.	`middleware.ts:1-21`
High	Room code generator (`create-client.tsx:31-34`) uses `Math.random()`, not `crypto`, over a 6-char/32-symbol space, with no uniqueness check against existing rooms at creation time.	`create-client.tsx:31-34`
High	Team Maker's "seeded" shuffle is a linear-congruential generator reseeded with `Date.now()` on every call — provides no reproducibility and has known modulo bias for small team sizes; should just use the `Math.random()` swap-shuffle used elsewhere in the codebase.	`tools/team-maker/page.tsx:53-90`
Medium	Inconsistent `searchParams` typing: `src/app/room/page.tsx` uses the pre-Next-15 synchronous shape while `src/app/room/[code]/page.tsx` correctly awaits the promise form — one of these is using a deprecated pattern.	`room/page.tsx:3-7` vs `room/[code]/page.tsx:5`

Severity	Bug	File:Line
Medium	Duplicate-message dedupe (`isDuplicateMessage`) compares `user_id + created_at + content` instead of the already-available message `id` ; two identical messages in the same millisecond would be incorrectly dropped.	`room-client.tsx:30-37`
Medium	Lucky Wheel's pointer-angle-to-winner math is a non-trivial inverse of the draw transform; plausible but unverified across all rotation quadrants — a mismatch here would silently declare the wrong winner.	`tools/lucky-wheel/page.tsx:337`
Low	Name Draw: once any duplicate name is drawn, all copies of that name are removed from the pool simultaneously (filter matches by string, not instance).	`tools/name-draw/page.tsx:85`

9. Security Issues

Severity	Issue	Evidence
Critical	**Identity and host status are entirely client-controlled.** User ID is `Math.random().toString(36)` stored in `localStorage` (`src/lib/room-user.ts:6-16`); host status is `localCreatorId === currentUser.id` read from `localStorage["spintra-room-creator-<code>"]` (`room-user.ts:49-54` , `room-client.tsx:118-120`). Any user can edit these values in devtools to impersonate another user or instantly become host.	`room-user.ts` , `room-client.tsx:118-120, 706-727`
Critical	**No verifiable server-side authorization.** Every host-gated action (lock room, change activity, manage participants) is enforced only by a client-side `isHost` boolean controlling JSX. Because no SQL/RLS migration files exist in-repo, it cannot be confirmed that Supabase Row Level Security actually restricts `UPDATE role='host'` on `room_participants` — if it doesn't, any anon-key holder can self-promote via a raw Supabase call, independent of this app's UI.	`room-client.tsx:453,528-570,706-727,1091+`
High	No rate limiting anywhere — chat messages, room creation, and join-by-code are all unthrottled from the client.	repo-wide grep, no matches
High	No length limits on chat messages or room names — unbounded growth/abuse vector.	`room-client.tsx:458` , `create-client.tsx:124-131`
Medium	No security headers configured (CSP, X-Frame-Options, HSTS, X-Content-Type-Options, Referrer-Policy).	`middleware.ts` , `next.config.ts`
Low	One `dangerouslySetInnerHTML` use, but content is a static string (E2E test hook) — not exploitable.	`src/app/create/page.tsx:16`
None found	No hardcoded secrets, no committed `.env` , no SQL injection surface (all queries parameterized), no hidden admin/debug panels.	(confirmed by two independent passes)

10. Performance Issues

- **Unbounded state**: `messages` and `participants` arrays in `room-client.tsx` grow without pagination or virtualization; `ScrollArea` renders every item on every render — will degrade in long-lived, active rooms.

- **3D hero scene not code-split**: `react-three-fiber` / `drei` / `three` are statically imported into the landing page bundle (`src/app/page.tsx:8,48`) instead of via `next/dynamic(..., { ssr: false })` ; no WebGL-unsupported, `prefers-reduced-motion` , or low-power device fallback, and no error boundary around `<Canvas>` .
- **No pagination** on initial chat history load (`room-client.tsx:846-850`) — fetches entire history unconditionally.
- Lucky Wheel redraws the full canvas path/text every animation frame during a spin with no memoization of the static wheel image between spins — a performance concern only at large (50+) entry counts.

11. Architecture Issues

- **No backend/API layer at all** — every data operation goes straight from the browser to Supabase using only the anon key. This collapses the entire security model onto RLS policies that don't exist in this repository, making the architecture's safety unverifiable and, per the security findings above, likely unsafe as currently evidenced.
- **Deprecated routing convention** (`middleware.ts` vs. Next 16's `proxy.ts`) — a framework-version-specific risk unique to this repo (per `AGENTS.md` 's own warning that this Next.js version has breaking changes from training-data assumptions).
- **Duplicated redirect logic** for `/room?code=X` exists independently in both `middleware.ts` and `room/page.tsx` — a maintenance hazard if the two ever diverge.
- `@supabase/ssr` **is a declared dependency but never used** anywhere in the codebase — either dead weight or a sign the intended SSR/server-auth architecture was never finished.
- **No error boundaries at the routing level** (no `error.tsx` / `not-found.tsx`) — any thrown error in a data-fetching path surfaces Next's default error page.

12. UI/UX Issues

- **FOUC on every load**: `src/app/layout.tsx:54` hardcodes `className="dark"` server-side; actual theme is applied only after client mount, causing a flash of the wrong theme for light-mode users.
- **Leaked feature-specific CSS overrides**: a "Light-Mode Global Contrast Fixes" block in `globals.css:240-347` hardcodes `!important` overrides for specific component classes (chat bubbles, sidebar tabs) rather than using the theme variable system already defined in the same file — fragile, and easy to silently miss when new components are added.
- **Hardcoded gradient colors** in `feature-card.tsx` / `navbar.tsx` bypass the `--gradient-*` CSS variables already defined — two sources of truth for the "brand gradient."
- **Duplicate "aurora" background implementations** (a CSS class and a canvas component) with independently hardcoded colors; only one is actually used.
- **Misleading "Live" status badge** under the BroadcastChannel fallback (see §6).
- **Silent data-load failures**: `loadParticipants` , `loadMessages` , and `loadRoomDetails` all fail with only a `console.error` — no toast, no retry affordance, leaving stale/empty UI with no user-visible explanation.
- **No loading skeletons** while initial room/chat/participant data is in flight.

13. Accessibility Issues

- No `aria-pressed` on any selection/toggle button across the 11 tool pages (dice type, category pickers, mode selectors) — screen-reader users get no indication of the active option.
- Lucky Wheel's hidden `<input type="color">` overlay has no visible keyboard-focus styling.
- Color is a primary (though not sole) differentiator for teams in Team Maker; palette is not color-blind-audited.

- (Positive) Icon-only buttons generally have proper `aria-label`s, and modal/dialog/sheet/drawer components inherit correct focus-trapping from Base UI/vaul rather than reinventing it.

14. Technical Debt

- Three separate, mutually inconsistent lint report artifacts (`eslint-report.json`, `lint-output.txt`, `tool-warnings.json`) are **committed to git** (458KB/30KB/342KB) and disagree on error/warning counts — evidence lint is run ad hoc, not via CI, and none of the three can be trusted as current.
- Sound-toggle boilerplate (state + icon button) duplicated verbatim across all 11 tool pages.
- `shuffleArray` (Fisher-Yates) duplicated identically in two files while a third (Team Maker) reinvents a weaker version instead of reusing it.
- README is unmodified `create-next-app` boilerplate — no architecture, setup, or environment documentation exists for a project with a non-trivial realtime/Supabase dependency.
- `@supabase/ssr` dependency installed and unused.
- Empty self-closing icon-tag formatting artifacts scattered across ~6 UI component files (cosmetic evidence of unreviewed generated code).

15. Code Quality Issues

- 16 real ESLint errors per the most recent lint snapshot, all `react-hooks/purity` / `react-hooks/set-state-in-effect` violations (e.g., `Math.random()` called during render in `hero-scene.tsx:73-75`; synchronous `setState` inside `useEffect` in `navbar.tsx:34`) — genuine React 19 compiler-rule violations with real correctness/perf implications.
- ~53-55 unused-variable warnings concentrated in `room-client.tsx` (12), `tools/tournament/page.tsx` (10), `tools/lucky-wheel/page.tsx` (9), `tools/team-maker/page.tsx` (4).
- 11+ `console.error` / `console.warn` calls left in production code paths with no structured logging/telemetry integration.
- `tsconfig.json` has `strict: true` (good), but there's no `typecheck` script wired into any workflow, so type errors are only caught implicitly at build time.

16. Testing Gaps

- **Exactly one automated test exists**: `tests/smoke.spec.ts` (Playwright, 19 lines) — covers create-room → join → host-badge assertion only.
- **Zero coverage** of all 11 standalone tool pages (coin-flip, dice, guess-number, lucky-wheel, name-draw, never-have-i-ever, rps, team-maker, tournament, truth-or-dare, would-you-rather).
- **Zero coverage** of the realtime sync logic, Supabase integration, host election, or chat in `room-client.tsx`.
- No unit/component test framework (Jest/Vitest/RTL) present at all — Playwright e2e is the only tooling, and `package.json` has no generic `test` script (only `test:smoke`).
- No CI to run even that one test automatically on every change.

---

17. High-Priority Fix Order

1. **Resolve the `middleware.ts` → `proxy.ts` deprecation risk** and confirm the room-join redirect actually still works under Next 16 (verify against `node_modules/next/dist/docs`).
2. **Establish real server-side authorization** — verify/lock down Supabase RLS policies for `room_participants` / `chat_messages` / `rooms` (host role writes, capacity, lock enforcement); do not trust `isHost` /localStorage for anything security-relevant.
3. **Fix room creation** so it actually persists to Supabase with a server-verified unique code, closing the "room never exists server-side" gap.
4. **Fix double-elimination tournament bracket** (losers-bracket advancement) or remove/hide the mode until implemented.
5. **Fix the theming FOUC** (adopt `next-themes` properly, since it's already a dependency, instead of the hand-rolled provider).
6. **Stand up CI** (lint + typecheck + `test:smoke` on every PR) and stop committing generated lint-report artifacts.
7. **Add rate limiting and input length limits** on chat/room-creation/join flows.
8. **Add security headers** (CSP, X-Frame-Options, HSTS, etc.) at the middleware/config layer.
9. **Write a real README** with environment variable documentation (`.env.example`) and Supabase setup steps; without this, no one else can run the app.
10. **Expand test coverage** beyond the single smoke test, prioritizing the room/realtime logic and the fixed double-elimination bracket.

18. Final Verdict

Not Production Ready

The UI and single-player feature layer are in reasonable shape, but the core multiplayer product has no real authentication or authorization boundary, an unpersisted room-creation flow, a broken headline feature (double-elimination), a Next-16-deprecated routing file, zero CI, and near-zero test coverage. None of these are cosmetic — they go to correctness, security, and the ability to safely operate the app at any real scale. This needs the fixes above (particularly #1–#5) before it should be exposed to real users.